

The image features the text "DL for ENG" in a bold, 3D, gold-colored font. The letters are floating on a vast, blue ocean with visible ripples and small waves. In the background, there are rolling blue hills or mountains under a sky filled with soft, grey clouds. The overall scene has a serene and expansive feel.

DL for ENG

LECTURE 12.2

Prediction of Power Grid Stability

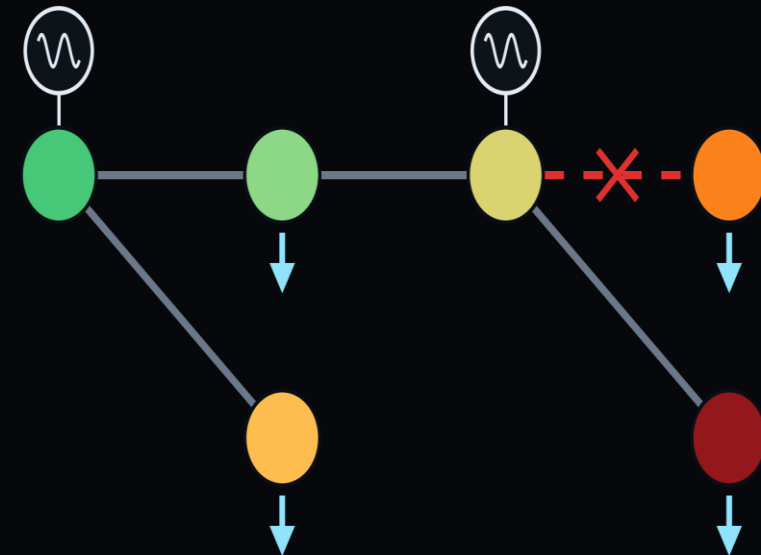
Estimation asked what the grid is doing. This asks where it is going — under a disturbance that is not in your training set, on a network that is not the one you trained on.

Remus Teodorescu

With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk · Deep Learning for Engineering · 2026

Transient stability · physics-informed GNN · N-1 screening · inference at the edge




predicted stability margin after the fault stable → critical

What You Will Be Able To Do

THE GOALS, IN PLAIN WORDS

- **distinguish** — the three questions people call stability, and say which one this is
- **explain** — why screening is combinatorial, and what a learned predictor is actually for
- **read** — the equal area criterion as an energy statement, and get a margin from it
- **state** — why the cases you must get right are the ones you have least of
- **represent** — a contingency as a change to the graph, not a change to a feature vector
- **argue** — why held-out topologies is the honest test, and held-out samples is not

WHAT TODAY ASSUMES

from L12.1	the swing equation, and the graph
from L5.1	message passing, and equivariance
from L6.2	compounding error in a rollout
from L7.2	why it needs two conditions

ESTIMATION WAS INTERPOLATION

Extrapolation now; the residual survives.

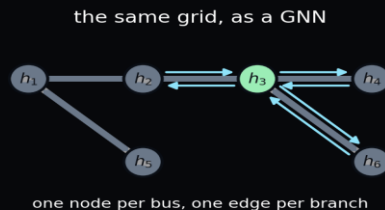
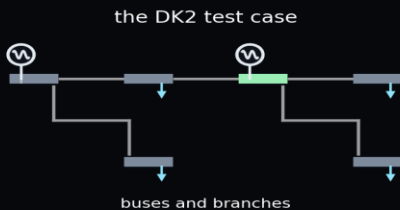
AND THE LAST LECTURE

It closes with what the course has been teaching.

Graph Networks, and Why Equivariance Matters Here

WHAT YOU ALREADY HAVE

- the same message-passing network as L12.1, now predicting forward in time
- equivariance is the property that is appropriate here** — permute the buses, the answer permutes
- weight sharing makes the update local, so it transfers to a grid it has not seen



THE NETWORK, CONCRETELY

inputs	pre-fault state, post-fault graph
hidden	message passing, shared weights
output	a stability margin from a trajectory
from L7.1	the swing residual in the loss

THE CONTINGENCY IS AN INPUT

Only expressible if the graph is an input.

AND THE RESIDUAL IS L7.1's

An ODE per machine, by autograd. New operator.

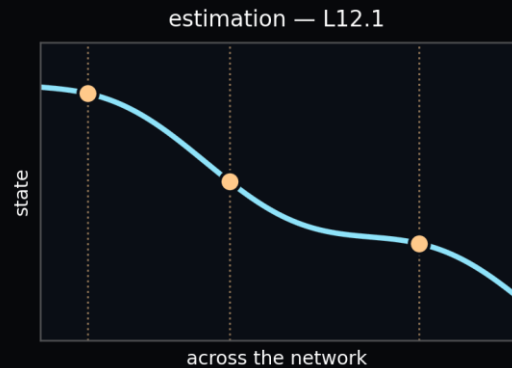
From What Is, to What Will Be

THE STATE YOU RECOVERED IS AN INITIAL CONDITION

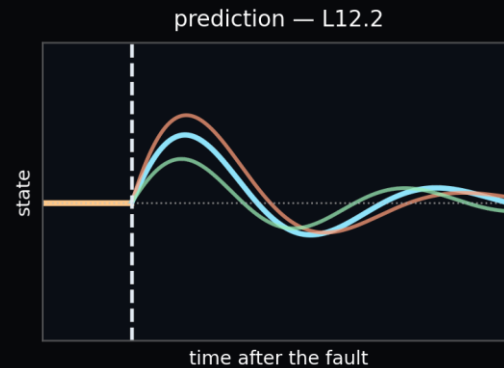
$$\mathbf{x}(t) = \{\delta_i(t), \omega_i(t)\}, \quad \mathbf{x}(0) = \hat{\mathbf{x}}_{L12.1}$$

δ_i rotor angle, ω_i speed, $\hat{\mathbf{x}}_{L12.1}$ the estimated state

- L12.1 produced a state consistent with the measurements and the physics



measurements pin it down, the physics fills the gaps



the data stops at $t = 0$, the physics carries the rest

WHAT CHANGES

question	where is it going, not where is it
data	stops at $t = 0$
physics	carries the whole prediction
error	compounds instead of averaging out
topology	changes — that is the point

WHY THE ESTIMATE MATTERS TWICE

A one per cent angle error at a instrumented bus is unremarkable in an estimate and can decide stable from unstable a second later. Carry L12.1's worst-bus error into this lecture, not its mean.

Rotor-Angle, Frequency and Voltage Stability

THREE QUESTIONS, THREE TIMESCALES

- **rotor angle** — do the machines stay in step after a fault? Seconds, governed by the swing equation
- **frequency** — does frequency stay within limits after a large imbalance? Seconds to minutes, inertia then reserves
- **voltage** — can the network hold its voltages after a disturbance? Minutes, reactive support and load recovery

WHAT THIS LECTURE PREDICTS

Rotor-angle stability in the first seconds after a fault — the large-disturbance case the classification calls TRANSIENT — with the classical machine model.

Frequency enters only through RoCoF, which shares the same inertia term as the swing equation. RoCoF and nadir are what protection acts on.

AND WHAT IT DOES NOT

Voltage stability needs load dynamics and reactive control the classical model does not contain. Say so in your report rather than letting a reader assume the prediction covers it.

Given This State and That Fault, What Happens?

A WELL-POSED QUESTION WITH AN EXPENSIVE ANSWER

$$\frac{2H_i}{\omega_s} \ddot{\delta}_i + D_i \dot{\delta}_i = P_{m,i} - P_{e,i}$$

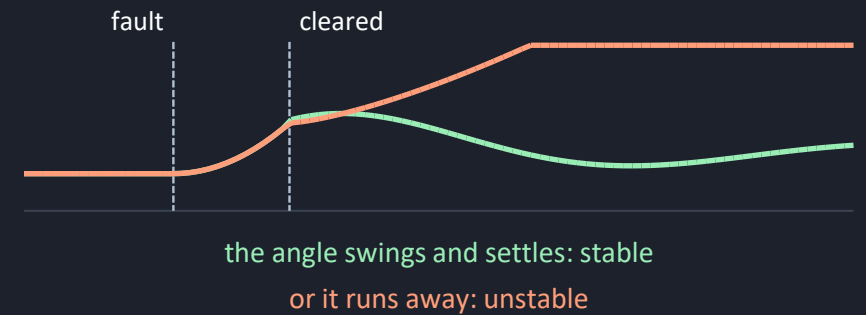
H_i inertia, D_i damping, ω_s synchronous speed, $P_{m,i} - P_{e,i}$ the power imbalance

- take the estimated state, apply a fault, clear it, integrate the swing equation
- the trouble is not formulating it
- the trouble is that an operator must ask it for every credible contingency, continuously
- everything in this lecture is a response to that cost

THE INGREDIENTS

x(0)	the estimated state, from L12.1
the fault	where, what type
clearing time	when protection acts
post-fault net	topology after the trip
the verdict	bounded, or not

A FAULT, AND WHAT FOLLOWS



Time-Domain Simulation, and Why It Is Not Enough

ALWAYS RIGHT, AND SOMETIMES TOO SLOW

- integrate the differential-algebraic system and you get the trajectory, to any accuracy you pay for
- the cost is per contingency and it does not amortise
- screening a large system against thousands of cases is not a real-time activity
- **so the industry runs a hierarchy** — cheap filters first, full simulation on survivors

WHAT SIMULATION GIVES YOU

accuracy	as much as the model deserves
detail	controls, protection, limits
trust	decades of validation
diagnosis	the whole trajectory, not a verdict

WHAT IT COSTS

per case	no reuse between contingencies
scaling	cases grow combinatorially
latency	too slow to sit inside a control loop

Why Screening Is a Combinatorial Problem

MORE QUESTIONS THAN THE GRID HAS PARTS

- N-1 security means surviving the loss of any single element
- on a few hundred branches that is already hundreds of cases
- multiply by fault location, type and clearing time, and by a state that changes every few minutes
- **a learned predictor is not there to beat integration on accuracy** — it is there to answer many cases at once

WHERE THE CASES COME FROM

single outages	branches, generators, transformers
fault type	three-phase, line-to-ground, ...
location	along the line, not just at the ends
clearing	nominal, and delayed by backup
operating point	and all of it again, next hour

THE HONEST SCOPE OF THE EXERCISE

You will screen single-branch outages on a small network, which is a few dozen cases and runs in minutes. The combinatorial argument is why the method matters at real scale; do not claim you demonstrated it at real scale.

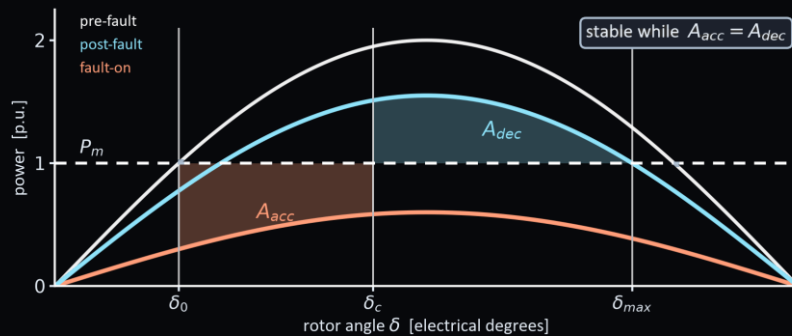
The Equal Area Criterion

WHY A MACHINE COMES BACK, OR DOES NOT

$$\int_{\delta_0}^{\delta_c} (P_m - P_e) d\delta = \int_{\delta_c}^{\delta_m} (P_e - P_m) d\delta$$

δ_0 pre-fault, δ_c clearing, δ_m maximum angle, P_m mechanical, P_e electrical power

- during the fault the machine accelerates, because electrical power collapses
- read as energy, the kinetic energy gained must be absorbable after clearing



$$t_{cc} = \max\{t_c: \delta(t) \text{ bounded } \forall t > t_c\}$$

WHY IT MATTERS FOR A LEARNED MODEL

It tells you what the answer depends on: inertia, pre-fault loading, the post-fault coupling, and the clearing time. Those are the features a predictor must see.

It also gives a sanity check with no network in it: if your predictor disagrees with the equal-area verdict on a single-machine case, it is wrong.

A TEST YOU CAN COMPUTE BY HAND

The one-machine-infinite-bus case has a closed-form answer. Notebook 01 keeps the habit for two machines: you sweep clearing times by simulation and check one label of the dataset before anything is learned.

From a Verdict to a Number

STABLE OR UNSTABLE IS NOT ENOUGH TO OPERATE ON

$$V(\delta, \omega) = \frac{1}{2} \sum_i M_i \omega_i^2 + V_{\text{pot}}(\delta)$$

V transient energy, M_i inertia coefficient, ω_i speed, V_{pot} the potential term

- a binary verdict tells an operator nothing about how close they are
- the energy formulation gives a margin instead
- positive means stable, and how positive says how much room is left

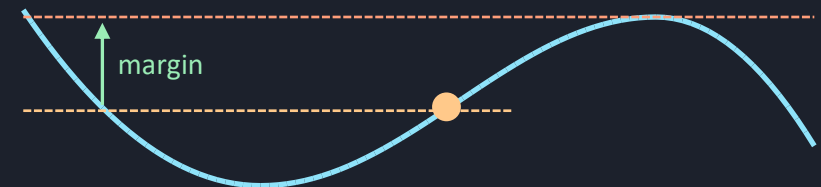
$$\eta = \frac{V_{\text{cr}} - V_{\text{cl}}}{V_{\text{cl}} - V_{\text{s}}}$$

V_{cr} critical energy, V_{cl} energy at clearing, V_{s} at the stable equilibrium

PREDICT THE MARGIN, NOT THE LABEL

label	stable or unstable, no gradient
margin	how much room is left
CCT	how much time is left
all three	derivable from a predicted trajectory

THE MARGIN, DRAWN



the crest: the energy that must not be reached
the ball: the energy when the fault clears

What a Purely Data-Driven Predictor Does

AND IT WORKS, UNTIL IT DOES NOT

$$p = g_{\theta}(\mathbf{x}_0, c) \in [0,1]$$

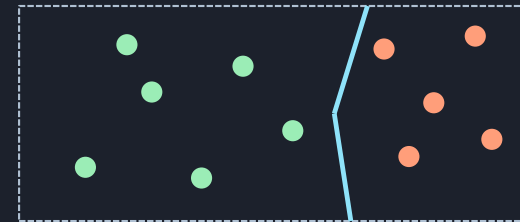
g_{θ} the learned model, $\mathbf{x}_0$ pre-fault state, c the contingency, p probability of stability

- simulate many cases, label them, train a classifier on features of the pre-fault state
- it performs very well, because the test cases came from the training distribution
- nothing in the model knows a swing equation
- it has learned a correlation, and correlations do not survive a topology it has not seen

WHAT IT GETS RIGHT

speed	a forward pass, milliseconds
in-distribution	genuinely high
simplicity	no solver in the loop
as a filter	a defensible first pass

THE FAILURE IS SILENT



the training set

a case from outside it gets an answer just as confident

The Case You Must Get Right Is the One You Lack

TRAINING SETS ARE MADE OF THINGS THAT HAPPENED

$$p_{\text{train}}(c) \neq p_{\text{test}}(c)$$

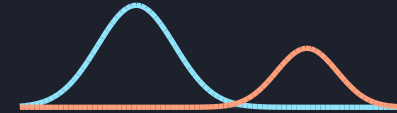
c the contingency, $p_{\text{train}}, p_{\text{test}}$ the two distributions over it

- grids are operated to be secure, so recorded history is overwhelmingly stable operation
- the cases you must get right are the ones you have least of
- **you can simulate them, and you should** — but then the training distribution is your simulator's
- a physics residual holds at states you never sampled, which is the point

THE ONE-SENTENCE ARGUMENT

Data tells you what has happened; physics tells you what can. A predictor built only from the first is confident exactly where it is least entitled to be.

what happened



what matters

WHERE THE SHIFT COMES FROM

new topology	an outage configuration not sampled
operating point	more wind, less inertia
rare fault	the one that matters, by definition
future grid	converter-dominated, unlike history

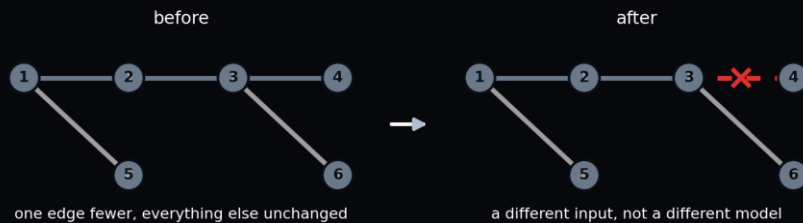
A Contingency Is a Change to the Graph

NOT A NEW PROBLEM — A NEW INPUT

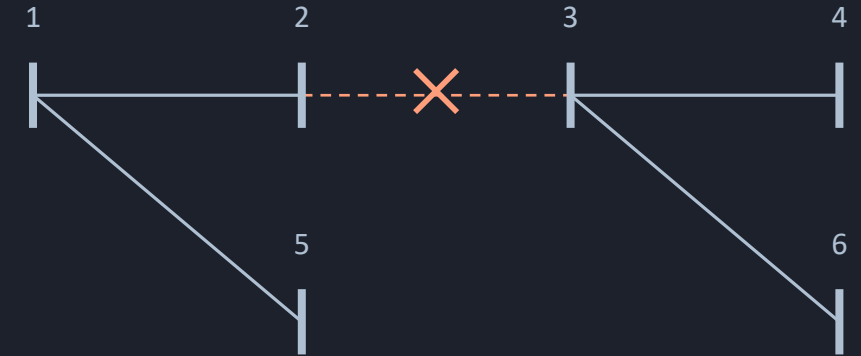
$$\mathcal{G} = (\mathcal{V}, \mathcal{E}), \quad \mathcal{G}_c = (\mathcal{V}, \mathcal{E} - \{e_c\})$$

$\mathcal{V}$ buses, $\mathcal{E}$ branches, e_c the branch the contingency removes

- removing a branch is an edge deletion
- if the model takes the graph as an argument, the post-fault network is a different input
- a model that flattens the state into a fixed vector cannot do this



THE SAME MODEL, A DIFFERENT GRAPH



one edge removed — no retraining

WHAT MUST STILL BE RECOMPUTED

$\mathbf{Y}$ changes with the topology, so the residual is rebuilt per contingency. Cheap and exact. What you avoid is retraining, not recomputing.

Message Passing, and the Property That Matters

THE RECAP, AND THEN THE POINT

$$h_{i^{(l+1)}} = \sigma \left(W_1 h_{i^{(l)}} + \sum_{j \in \mathcal{N}(i)} W_2 h_{j^{(l)}} \right)$$

$h_i^{(l)}$ node features, $\mathcal{N}(i)$ the neighbours of i , W_1, W_2 shared weights, σ nonlinearity

- each node updates from its own features and an aggregate over its neighbours
- **the property that is appropriate here is equivariance** — permute the buses, the prediction permutes with them
- weight sharing follows from the same structure
- the learned update is local and applied everywhere, so it transfers between grids

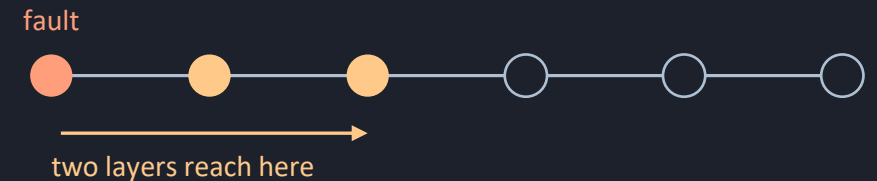
$$g_\theta(P\mathbf{x}, P\mathcal{G}) = P g_\theta(\mathbf{x}, \mathcal{G})$$

P a permutation of the buses, g_θ the learned model

THE MAPPING

node	a bus, or a machine
node features	δ, ω, P_m, H at that bus
edge	a line, present or tripped
edge features	conductance and susceptance
layers	how far influence travels per step

DEPTH IS A PHYSICAL CHOICE



too few layers: a distant machine never feels the fault
too many: every bus sees the whole grid

Physics-Informed Graph Network

THREE TERMS, AND WHY EACH IS NECESSARY

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_{\text{ph}} \mathcal{L}_{\text{swing}} + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}}$$

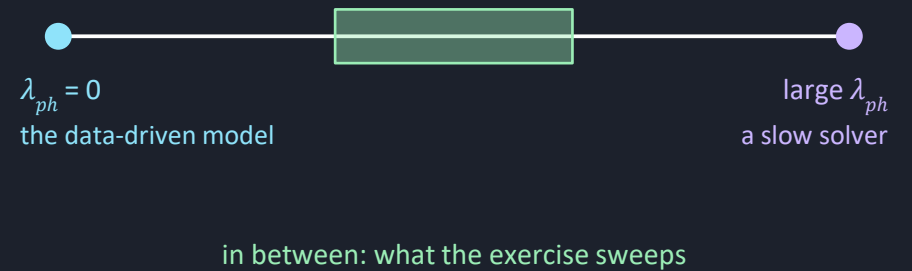
$\mathcal{L}_{\text{data}}$ the fit, $\mathcal{L}_{\text{swing}}$ the physics, $\mathcal{L}_{\text{ic}}$ the initial condition, λ their weights

- the data term fits simulated trajectories where you have them
- the physics term demands the prediction satisfy the swing equation
- the data term alone gives the data-driven predictor, better structured
- the residual is what survives distribution shift

WHAT EACH TERM BUYS

data	speed, and the shape of typical cases
physics	behaviour where data runs out
initial condition	built in, not a loss term
the weights	your statement of relative trust
together	auditable, and fast

THE WEIGHTS ARE THE EXPERIMENT



Writing the Residual on a Graph

THE SWING EQUATION, EVALUATED WHERE YOU LIKE

$$\mathcal{L}_{\text{swing}} = \frac{1}{NT} \sum_i \sum_k \left\| \frac{2H_i}{\omega_s} \ddot{\delta}_i + D_i \dot{\delta}_i - P_{m,i} + P_{e,i} \right\|^2$$

N machines, T collocation times, H_i inertia, D_i damping, $P_{e,i} - P_{m,i}$ the power imbalance

- one residual per machine per collocation time
- time derivatives come from autograd, exactly as in L7
- the electrical power term is where the graph enters — it sums over post-fault neighbours
- and collocation points should not all sit in the easy part of the window

TWO WAYS TO GET THE DERIVATIVES

continuous	network of t ; autograd gives $\delta, \dot{\delta}$
rollout	discrete step; residual on the update
continuous, pro	exact derivatives, free collocation
rollout, pro	matches how it will be deployed

WHERE THE RESIDUAL IS EVALUATED



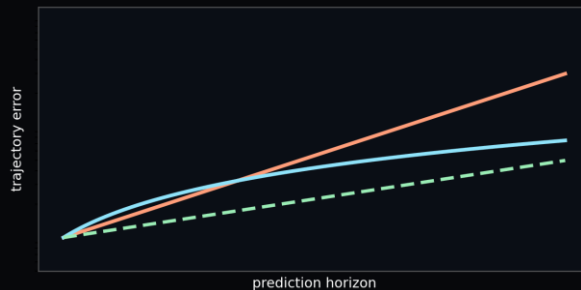
One Shot, or Step by Step

AND THE ERROR BEHAVES DIFFERENTLY IN EACH

$$\hat{\mathbf{x}}_{k+1} = \hat{\mathbf{x}}_k + \Delta t g_{\theta}(\hat{\mathbf{x}}_k, \mathcal{G}_c)$$

$\hat{\mathbf{x}}_k$ the predicted state, Δt the step, $\mathcal{G}_c$ the post-fault graph

- a rollout applies a learned update repeatedly, feeding its own output back
- **it matches the physics and it compounds its own errors** — L6.2's problem, on a trajectory
- a one-shot model maps the initial state and the contingency straight to the answer



PICK BY HORIZON

short horizon	either works
long horizon	rollout drifts without physics
margin only	one-shot is enough
full trajectory	rollout, with the residual on

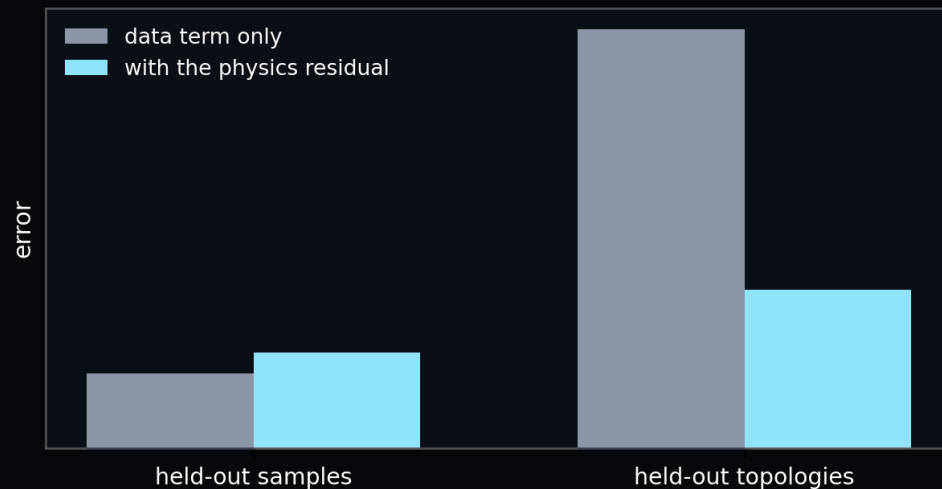
TEST TO FAILURE, NOT TO SUCCESS

Do not report the horizon at which your model works. Report the horizon at which it stops working, for each variant. The second number is informative and the first is a choice of where to stop plotting.

Held-Out Topologies, Not Held-Out Samples

SPLIT THE DATA THE WAY THE CLAIM REQUIRES

The usual random split tells you nothing here. Training and test cases drawn from the same set of topologies share the thing you claim to generalise over, and a high score means only that the model interpolates within it.



THE FOUR-WAY COMPARISON

MLP, seen	the flattering number
MLP, unseen	usually collapses
GNN, unseen	Ex_12.2: 57 ms off
PIGNN, unseen	the claim, tested
simulation	the truth, for all of them

AN HONEST NEGATIVE IS STILL A RESULT

Ex_12.2 measures the GNN row: with one line held out it is 57 ms off, on the unsafe side. It builds no PIGNN. If yours does not beat that, say so and find out why. A reported negative is worth more than a tuned positive.

What to Build In Rather Than Penalise

THE THIRD TIME THIS IDEA APPEARS IN THE COURSE

$$\omega_i = \omega_s + \Delta\omega_{\max} \tanh(\hat{\omega}_i)$$

$\hat{\omega}_i$ the raw network output, ω_s synchronous speed, $\Delta\omega_{\max}$ the bound imposed

The initial condition is known exactly — it is the estimate you start from — so impose it by construction rather than by a loss term, exactly as L7.2 imposed the initial condition on the heat equation.

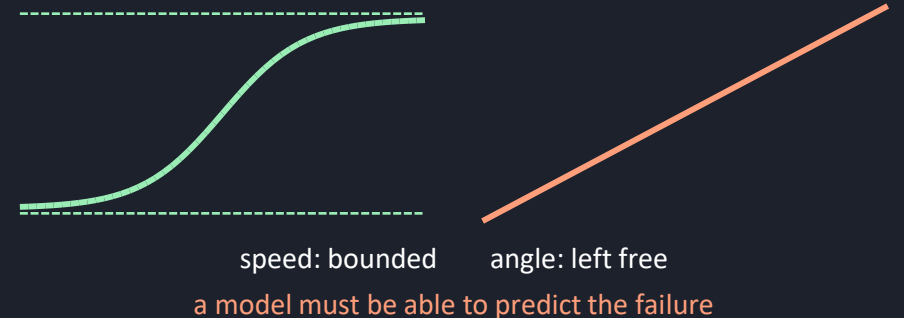
Speed deviations have a physical range; a machine at fifty per cent over synchronous speed has left the model's validity long before it left the optimiser's search space. Bounding the output keeps predictions interpretable when they are wrong.

Angle differences are the one to leave free. Bounding them would prevent the model from ever predicting loss of synchronism — which is the event you built the whole thing to see.

HARD, SOFT, FREE

initial condition	hard — build into the trial form
speed range	hard — bounded activation
swing equation	soft — the residual, weighted
angle separation	free — never bound it

THE MISTAKE THIS PREVENTS



How This Fails

FOUR, AND THE FIRST TWO ARE SILENT

- **confident off-distribution** — an unfamiliar case returns a smooth, plausible, wrong trajectory, with no warning
- **residual satisfied, answer wrong** — the model omits governors and protection, so a wrong answer can satisfy it
- **rollout drift** — each step is accurate and a hundred are not, visible only if you test past your horizon
- **the initial condition was wrong** — everything starts from L12.1's estimate, and an error there becomes a verdict here

THE ONE DIAGNOSTIC YOU HAVE

Evaluate the physics residual at inference, not only during training, and treat a large value as a refusal. A physics-informed model can police itself this way. A classifier cannot.

WHY THE FIRST TWO ARE SILENT

Neither announces itself. The output is smooth and plausible, so nothing in the prediction tells you which regime you are in.

AND THEY FAIL TOGETHER

No amount of care in this lecture recovers an estimation error made in L12.1.

Why Any of This Should Run Locally

TOO LATE IS NOT A PREDICTION

Transient stability plays out in the first seconds after a fault. A verdict that reaches a control room after the machines have already separated is a post-mortem, however accurate it is.

Central assessment involves getting measurements to a control centre, computing, and getting a decision back. Each leg costs time, and the total is what must fit inside the window the physics allows — not the compute alone.

Local inference removes the two communication legs. That is the entire argument for the edge here: not that the hardware is fast, but that the distance is short.

It also changes what can fail. A local predictor keeps working when the link to the control centre does not, which is exactly the condition under which grids get interesting.

WHAT LOCAL BUYS

latency	two comms legs removed
availability	survives loss of the link
bandwidth	send verdicts, not raw streams
privacy	measurements stay in the substation

AND WHAT IT COSTS

A model in a substation is a model somebody must update, version and validate in the field, on hardware in an electrically hostile place with a ten-year service life. The engineering problem moves; it does not disappear.

Where the Milliseconds Actually Go

ADD THEM UP BEFORE CHOOSING ANY HARDWARE

$$t_{\text{tot}} = t_{\text{meas}} + t_{\text{comm}} + t_{\text{inf}} + t_{\text{act}}$$

t_{meas} measurement, t_{comm} communication, t_{inf} inference, t_{act} actuation, t_{cc} clearing

Four terms. Measurement — a PMU reporting at 50 Hz cannot tell you anything sooner than its reporting interval, whatever follows it.

Communication. Inference. Actuation, which is a physical device closing or opening.

For a small network the inference term is the smallest of the four by a wide margin, and it is the only one people benchmark. That is the trap: optimising the term that was never binding.

The exercise asks you to measure all four on your own setup and present them as a budget. Only then does the question of hardware become answerable, because only then do you know how much time you were trying to buy.



indicative proportions only — inference is the thin one

THE CONSTRAINT

$$t_{\text{tot}} \ll t_{\text{cc}}$$

The total must be comfortably inside the critical clearing time. That is the link between the physics and the engineering.

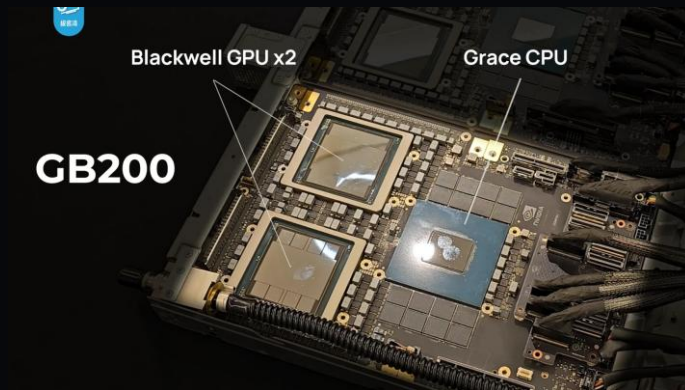
THE L11.1 PARALLEL

Blind distance was $d = v/f$: how far the car travelled between seeing and acting. The same quantity for a grid, measured rather than assumed.

Jetson Thor, and What It Is Actually For

THE BOARD IN THE LAB

- **based on GB200** — the same GPU plus CPU Blackwell architecture
- **for physical autonomous systems** — robots, self-driving cars
- **local edge AI for grid control** — power flow for hundreds of nodes



THE JETSON THREAD, CLOSED

compute	2 070 TFLOPS sparse FP4, Blackwell
CPU	14-core Arm Neoverse V3AE
memory	128 GB LPDDR5X, 273 GB/s
power	40 to 130 W, configurable
what it is for	many contingencies at once, on site

NOT WHAT YOU WOULD DEPLOY

A substation device is a hardened IED with a decade of service life and IEC 61850 interfaces — not a development kit drawing over a hundred watts. Thor is what you learn on; say which is which.

Getting a Trained Model onto the Board

AND WHAT CHANGES WHEN YOU DO

$$r = \frac{1}{t_{\text{inf}}} \geq N_c f_{\text{screen}}$$

r throughput, t_{inf} inference time, N_c contingencies to screen, f_{screen} the refresh rate

Training happens on a workstation; inference happens on the board. Between them sits an export step and an optimisation step, and both can change numerical behaviour — reduced precision first among them.

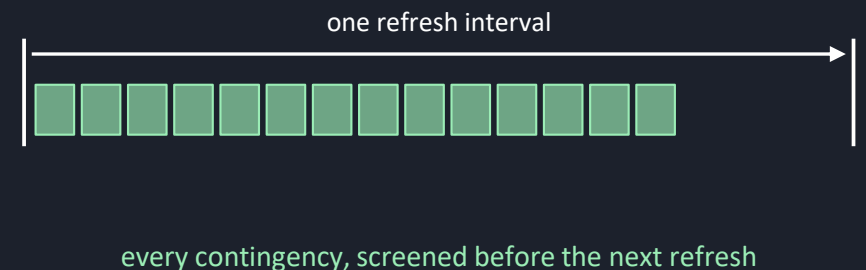
Screening is naturally batched: many contingencies, the same weights, independent graphs. Throughput matters as much as single-case latency. The requirement is to screen the whole list within the interval you refresh it.

Check the physics after optimising, not just the accuracy. If quantisation degrades the residual on cases the model handled before, you have traded correctness for speed you may not have needed — and the residual will tell you, where a test-set score might not.

THE STEPS

export	out of the training framework
optimise	for the target device
precision	reduced, then re-check
batch	many contingencies at once
measure	latency and throughput, on device

THE WHOLE LIST, IN ONE INTERVAL



What Belongs Local, and What Does Not

NOT A REPLACEMENT FOR THE CONTROL CENTRE

A local predictor sees local measurements and a local model. It cannot know the state of the whole system, it cannot be retrained in the substation, and it should not be making decisions that require a system-wide view.

What it can do is act fast on the cases where speed is the binding constraint, and flag rather than decide when its own residual says it is outside what it understands. Refusal is a legitimate output and should be designed in from the start.

The control centre keeps the full model, the training, the validation and the authority. This division — fast and local, careful and central — is older than machine learning in this industry, and the new component fits into it rather than replacing it.

LOCAL

inference	on the current window
residual check	am I outside what I know
fast flags	when the budget is tight
buffering	keep data when the link drops

CENTRAL

training	and every retraining
validation	against full simulation
system view	coordinating across substations

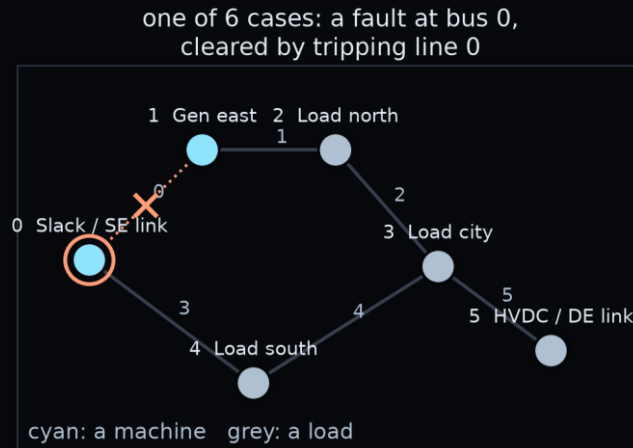
Ex_12.2 — What You Will Build

ONE NOTEBOOK, ON COLAB

You begin from the grid of Ex_12.1 and simulate a contingency set: 180 operating points, six contingencies, a critical clearing time for each. The simulation is the truth, and section 4 measures what it costs.

Section 5 is the graph network. You write its layer and its loss, train it on 144 operating points and test it on 36 it has not seen.

Section 6 holds a whole topology out, the fair test, and turns the prediction into a screen by choosing a threshold.



THE NOTEBOOK, BY SECTION

- | | |
|-------|------------------------------------|
| 1 – 3 | contingencies, the swing equation |
| 4 | the simulation, and a coarser step |
| 5 | the graph network — TODOs |
| 6 | an unseen line, and the threshold |
| 7 | what the notebook says |
| 8 – 9 | your report, the mini projects |

THE QUESTION THAT CARRIES THE MARKS

Would you let this run unsupervised in a substation? The answer is no, and the marks are for saying what would have to be true first.

The Orderings That Must Come Out Right

THE RELATIONS, NOT THE NUMBERS

On graphs it has seen, the network is within a few milliseconds on operating points it has not. That is the control, and a result that disagrees with it means something is wrong with the implementation.

On a held-out topology it degrades sharply, and on the unsafe side. A PIGNN, not built here, is the method meant to do better. The size of the gap depends on your setup; its direction should not.

On the board, inference should be a small fraction of the total budget. If it is not, either your model is far larger than the problem needs or your measurement includes warm-up.

Add a physics term and its residual should rise where accuracy falls. If it does not, the term is not constraining what you think it is — worth finding out in a notebook rather than in a substation.

THE CHECKLIST

one case	20 ms decide it
coarser step	cheaper; verdicts change
seen graphs	within a few ms, no misses
held-out line	far off, on the unsafe side
threshold	raised until none is missed
latency	inference is the thin slice

ANY ONE OF THEM BACKWARDS

Six relations. A reversed ordering points at the implementation rather than at the physics, and that is the fastest debugging tool in the exercise.

What This Method Is Not Ready To Do

STATED PLAINLY, BECAUSE SOMEBODY WILL ASK

- **not a replacement for simulation** — it is a filter; simulation stays the answer of record
- **it inherits the model's limits** — no exciter, no governor, no protection, so the residual cannot see those failures
- **it carries no guarantee** — no covariance, no bound, no certificate; the residual is evidence, not proof
- **not tested at scale here** — a few dozen buses is a teaching case; the real argument lives at hundreds or thousands

WHAT IT IS FOR, THEN

Screening. Answering the same question thousands of times more cheaply than integration can, so integration is spent where it matters. A screening tool that is wrong occasionally is useful; a tool of record that is wrong occasionally is not.

THE QUESTION TO ASK ANYONE

Ask what exactly was verified, and over what set.

AND THE STANDARD FOR YOUR REPORT

State the boundary of your claim first.

Questions

1 – 2 · TRANSIENT STABILITY

1 Which stability question is asked here, and why is screening combinatorial?

Rotor-angle stability: after a fault, do the machines stay in step? Every operating state times every possible outage is a case, and each needs a simulation in time, so the cases outgrow the time available. A learned predictor screens them and leaves the simulation for the doubtful ones.

Kundur ch. 2, 13; Ex_12.2 sections 1 – 4

2 What does the equal area criterion say, and why is a margin worth more than a verdict?

Read as energy: the machine stays in step if the area that can slow it down after the fault is cleared is at least the area that sped it up during the fault. The difference is a margin, which tells an operator how close a case is; a verdict says only stable or not. *Kundur ch. 13; Ex_12.2 section 3*

3 – 4 · GRAPH PREDICTOR

3 Why is a contingency a change to the graph, and what does a graph network gain from that?

A tripped line is an edge removed, not a new value in a feature vector. A graph network applies the same local rule on whatever graph it is given, so it can be tested on topologies it has never seen, which a dense network cannot represent at all. *Ex_12.2 section 5; L5.1*

4 Why hold out topologies and not samples, and what is the method not ready to do?

Held-out samples of topologies seen in training measure interpolation and flatter a model with no physics in it; the unstable cases are the rare ones, and the ones that must be right. Held-out topologies are the fair test. The predictor screens; it is not a tool of record. *Ex_12.2 section 6*

What You Have Actually Been Learning

ONE METHOD, FIVE FIELDS

L8 heat	a plate, steady then transient
L9 flow	a cavity, a cylinder, then turbulence
L10 electrochemistry	a cell, then a stack, then an optimisation
L11 robotics	a policy, then constraints and energy
L12 the grid	a state, then a prediction

THE ARC, IN ONE SENTENCE

Part 1 taught you what a network is. Part 2 taught you to put a physical law where the labels would go.

AND ONE STRUCTURE, THROUGHOUT

a network	proposes a field, or a decision
a residual	judges it against the physics
a constraint	built in, not penalised
an objective	when you want more than a solution

AND WHAT YOU SHOULD BE SCEPTICAL OF

Every lecture in Part 2 named a case where a classical method is better. That list is the useful one.

WHERE IT GOES NEXT

L13 takes five of these fields further, as mini-projects where the answer is not known in advance.

One Line to Remember

Data tells you what has happened. Physics tells you what can. The case you must get right is, by construction, the one you have no data for.

AND THREE THINGS THAT FOLLOW

Test on held-out topologies, not held-out samples. Any other split measures interpolation and calls it generalisation.

Let the model refuse. A physics residual evaluated at inference is a self-check no purely data-driven predictor has, and it is the difference between a demonstration and something deployable.

Measure the latency budget before choosing hardware. The term everyone benchmarks is rarely the one that binds.

IF YOU DO ONE THING

Build the four-way table on held-out topologies. It is a day's work, it is the experiment the whole lecture argues for, and it will tell you whether the physics term earned its place in your setup rather than in a paper's.

A negative result there, honestly reported, is a better piece of engineering than a tuned positive.

References and Further Reading

POWER SYSTEM DYNAMICS

Kundur (1994), Power System Stability and Control — transient stability, the equal area criterion, machine models. Chapter 13 is the one for this lecture.

Machowski, Bialek & Bumby, Power System Dynamics — a modern treatment with more on control and protection.

Pai, Energy Function Analysis for Power System Stability — the energy-margin formulation used here.

LEARNING, AND THE GRAPH

Raissi, Perdikaris & Karniadakis (2019), J. Comput. Phys. 378, 686–707 — the physics-informed formulation used all course.

The graph network material from Part 1 of this course — message passing, and why equivariance matters.

The physics-informed GNN literature for power systems is active and young. Reproduce before you cite, and be sceptical of any accuracy figure reported without a held-out-topology split.

DATA AND TOOLS

Energinet	Danish injections and exchanges
Fingrid, SvK	Nordic frequency, real events
ENTSO-E	pan-European operational data
PyPSA-Eur	open European network topology
IEEE test cases	small and exact, for verification
NVIDIA Jetson	the board in the lab

Everything in the exercise runs on torch, numpy, scipy and matplotlib. The graph layer is written by hand, so no graph library is needed.

END OF THE COURSE

Applied PINN for Energy

From a Poisson equation on a unit square to a stability predictor on a national grid, with the same four moves throughout. What you take away is the habit, not the architecture.

TO FINISH

Ex_12.2	the held-out table, and the latency
the report	state the boundary of your claim
MiniProject	turbulence closure, if you took L13
afterwards	apply it outside this course

The most useful thing you can do next is take a problem you were not set, and ask what physics you already trust about it.